

Unit-2 Software Development Life Cycle (SDLC)

Introduction to SDLC

- The SDLC is a structured process used for developing high-quality software.
- Ensures software meets customer expectations, is delivered on time, and cost-efficient.
- Provides a well-defined plan for software development with distinct phases.

Phases of SDLC

- 1. Planning
- 2. Requirement Analysis
- 3. Design
- 4. Implementation (Coding)
- 5. Testing
- 6. Deployment
- 7. Maintenance
- Each phase has specific deliverables and is critical for success.

Planning Phase

- Initial phase defining the project's scope.
- Key activities:
 - - Identifying objectives
 - - Feasibility study (technical, operational, financial)
 - - Resource planning and budgeting
- Creates roadmap for development.

Requirement Analysis

- Gather and document business needs.
- Interview stakeholders for functional and non-functional requirements.
- Output: Software Requirement Specification (SRS) document.
- Foundation for design and development.

Design Phase

- Convert SRS into design plan.
- Divided into:
 - - High-Level Design (System Architecture)
 - - Low-Level Design (Component Design)
- Includes interface designs, data models, system behavior.

Implementation / Coding

- Developers write code as per design.
- Activities include:
 - - Choosing languages/tools
 - - Version control and code review
 - - Ensuring code quality and standards.

Testing Phase

- Ensures software is bug-free and meets requirements.
- Types of testing:
 - - Unit Testing
 - - Integration Testing
 - - System Testing
 - - UAT
- Bugs are reported, fixed, and re-tested.

Deployment

- Release software into production.
- Activities:
 - - Installation
 - - Configuration
 - - User training and support
- May be phased or full release.

Maintenance

- Post-deployment software upkeep.
- Includes:
 - - Bug fixing
 - - System upgrades
 - - Performance improvements
 - - Adapting to changing needs.

SDLC Models

- Different execution models:
 - - Waterfall
 - - Agile
 - - Spiral
 - - V-Model
 - - Iterative
- Choice depends on requirements and risks.

Comparison of SDLC Models

- Waterfall: Flexibility - Low, Risk - Low, Documentation - High
- Agile: Flexibility - High, Risk - Medium, Documentation - Low
- Spiral: Flexibility - High, Risk - High, Documentation - Medium
- V-Model: Flexibility - Low, Risk - Medium, Documentation - High

Benefits of SDLC

- Structured approach
- Risk and resource management
- Improved product quality
- Predictable timelines and cost
- Facilitates monitoring and control.

Challenges in SDLC

- Incomplete/misunderstood requirements
- Poor stakeholder communication
- Scope creep
- Insufficient testing/maintenance
- Wrong model selection

Conclusion

- SDLC builds robust, scalable, quality software.
- Right model + rigorous process = project success.
- Continuous improvement is key.

Q&A

- Thank you for your attention!